Project 2 Report

Design

이번 과제는 xv6에서 kernel-level thread를 구현하는 것이다. 기존 xv6는 `struct proc` 하나에 execution 컨텍스트와 주소 공간이 묶여 있었지만, 이를 분리하여 스레드들이 같은 주소 공간을 공유하되 실행 흐름은 독립적으로 갖도록 만드는 것이 목표다. 커널 내에는 `clone()` 시스템 콜 하나만 존재하고, CLONE_VM 비트로 주소 공간의 공유 여부를 구분지어 `fork()`로 새로운 프로세스를 생성할지, `pthread_create()`로 새로운 스레드를 생성할지 결정한다.

이를 구현하기 위해 명세에 제시된 대로 3단계로 나누어 생각해봤다.

Implementation & Code Analysis

Step 1: Extend Memory Management

먼저 `kernel/proc.c`에서 `mm_get()`, `mm_put()`, `find_free_tf_va()` 함수를 모두 구현하였다.

```
118  void
119  mm_get(struct mm_struct *mm)
120  {
121    acquire(&mm->lock);
122    mm->refcount++;
123    release(&mm->lock);
124  }
```

`mm_get()` 함수는 `refcount`를 1 증가시키는 함수다.

```
126  void
127  mm_put(struct mm_struct *mm)
128  {
129    acquire(&mm->lock);
130    mm->refcount--;
131    if (mm->refcount == 0) {
132      pagetable_t pagetable = mm->pagetable;
133      uint64 sz = mm->sz;
134      release(&mm->lock);
135
136      proc_freepagetable(pagetable, sz);
137      kfree((void *)mm);
138    } else {
139      release(&mm->lock);
140    }
141  }
```

`mm_put()` 함수는 `refcount`를 1 감소시키는 함수다. 추가로 `refcount`가 `0`이 되었

을 때 `pagetable`과 `mm` 구조체를 free해주는 기능을 구현하였다.

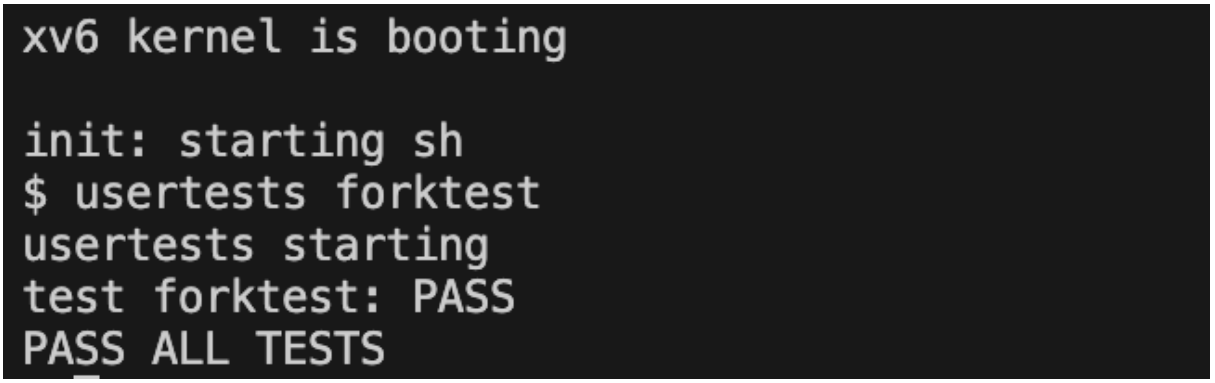
```
143  uint64
144  find_free_tf_va(pagetable_t pagetable)
145  {
146    struct proc * p;
147    for(int i = 0; i < NTHREAD; i++) {
148      uint64 va = TRAPFRAME - i * PGSIZE;
149      int used = 0;
150
151      for(p = proc; p < &proc[NPROC]; p++) {
152        if(p->state != UNUSED && p->mm != 0 && p->mm->pagetable == pagetable && p->tf_va == va) {
153          used = 1;
154          break;
155        }
156      }
157      if(!used) return va;
158    }
159    return 0;
160  }
```

`find_free_tf_va()` 함수는 사용되지 않는 trapframe의 virtual address를 찾는 함수다. 먼저 outer for문에서 슬롯 `0`번(`TRAPFRAME`)부터 `NTHREAD - 1`번 슬롯까지 순회하고, inner for문에서 현재 `proc` 테이블을 순회하며 해당 주소를 다른 스레드가 쓰고 있는지 검사한다. 이 중 사용되지 않는 슬롯이 있다면 해당 주소를 반환하고, 모두 사용 중인 경우 `0`을 반환한다.

```
210  static void
211  freeproc(struct proc *p)
212  {
213    if (p->trapframe) {
214      if(p->tf_va != 0 && p->tf_va != TRAPFRAME && p-> mm != 0 && p->mm->pagetable != 0) {
215        uvmunmap(p->mm->pagetable, p->tf_va, 1, 0);
216      }
217      kfree((void *)p->trapframe);
218      p->trapframe = 0;
219    }
```

`freeproc()` 함수에서 스레드 종료 시 그룹 리더가 아닌 경우 할당된 trapframe virtual address 매핑을 `uvmunmap`으로 해제하는 부분을 추가하였다.

여기까지 구현하였다면 메모리 누수(memory leak)가 해결되었을 것이다. 이 부분부터 확인해봤다.



Step 2: Implement Core Threading Primitives

본격적으로 스레딩에 필요한 기능을 시스템 콜 함수에 구현하는 단계다.

```
int
kclone(uint64 fn, uint64 arg, uint64 stack, int n_pages, in
t flags)
{
    // ...
    if (flags & CLONE_VM) {
        // === thread path ===
        np->mm = p->mm;
        mm_get(np->mm);

        np->group_leader = p->group_leader;
        np->tgid = p->tgid;

        acquire(&p->mm->lock);
        np->tf_va = find_free_tf_va(np->mm->pagetable);
        if(np->tf_va == 0) {
          release(&p->mm->lock);
          freeproc(np);
          release(&np->lock);
          return -1;
        }
        if(mappages(np->mm->pagetable, np->tf_va, PGSIZE,
(uint64)np->trapframe, PTE_R | PTE_W) < 0) {
          release(&p->mm->lock);
```

```
            freeproc(np);
            release(&np->lock);
            return -1;
          }
          release(&p->mm->lock);

          *(np->trapframe) = *(p->trapframe);
          np->trapframe->a0 = arg;
          np->trapframe->epc = fn;
          np->trapframe->sp = stack + n_pages * PGSIZE;
          np->ustack = (void*)stack;
        } else {
        // ...
```

`kernel/proc.c`의 `kclone()` 함수에서 스레딩 조건 분기를 추가하였다. `CLONE_VM` 비트가 설정된 경우 부모의 `mm`을 공유(`mm_get`)하고, `find_free_tf_va`를 통해 비어 있는 trapframe virtual address를 할당받는다. 스택 포인터는 인자로 받은 `stack + n_pages * PGSIZE`로 설정하였다.

```
418  int
419  kjoin(uint64 stack_addr)
420  {
421    struct proc *p = myproc();
422    struct proc *pp;
423    int havekids, pid;
424
425    acquire(&wait_lock);
426    for(;;) {
427      havekids = 0;
428      for(pp = proc; pp < &proc[NPROC]; pp++) {
429        if(pp->group_leader != p->group_leader || pp == p) continue;
430
431        havekids = 1;
432        acquire(&pp->lock);
433        if(pp->state == ZOMBIE) {
434          pid = pp->pid;
435          if(stack_addr != 0 && copyout(p->mm->pagetable, stack_addr, (char *)&pp->ustack, sizeof(pp->ustack)) < 0) {
436            release(&pp->lock);
437            release(&wait_lock);
438            return -1;
439          }
440          freeproc(p: pp);
441          release(&pp->lock);
442          release(&wait_lock);
443          return pid;
444        }
445        release(&pp->lock);
446      }
447      if(!havekids || killed(p)) {
448        release(&wait_lock);
449        return -1;
450      }
451      sleep(chan: p->group_leader, lk: &wait_lock);
452    }
453  }
```

`kjoin()` 함수는 같은 스레드 그룹 내의 형제 스레드가 종료될 때까지 대기하였다가 종료된 스레드의 자원을 회수하는 역할을 한다.

```
if(pp->group leader != p->group_leader || pp == p) continu
e;
    // ...
if(pp->state == ZOMBIE) {
```

현재 `proc` 테이블을 순회하며 호출한 스레드와 동일한 그룹 리더를 공유하면서(반대로 본인이거나 같은 그룹 소속이 아니면 무시하고 속행), 현재 상태가 종료된 `ZOMBIE` 상태인 형제 스레드가 있는지 검사한다.

```
if(stack_addr != 0 && copyout(p->mm->pagetable, stack_addr,
(char *)&pp->ustack, sizeof(pp->ustack) < 0) {
```

User-level 스레드 라이브러리(`uthread.c`)에서 스레드를 생성할 때 `malloc()`으로 할당했던 user 스택 영역은 커널이 자동으로 해제해주지 않기 때문에 `kjoin()`은 종료된 스레드의 구조체에 저장되어 있던 user 스택의 시작 주소(`pp->ustack`)를 사용자가 인자로 전달한 주소 공간(`stack_addr`)으로 안전하게 복사(`copyout`)해준다.

```
freeproc(pp);
```

위의 과정이 모두 완료되면 스레드의 개별 kernel 스택, `tf_va` 매핑 해제, `proc` 테이블 슬롯을 `UNUSED`로 되돌리는 작업을 수행하는 `freeproc(pp)`를 호출하여 자원을 회수한다.

```c
#define CLONE_VM 0x0100

int
thread_create(void (*fn)(void *), void *arg, int n_pages)
{
  void *stack = malloc(n_pages * PGSIZE);
  if(stack == 0) return -1;

  int pid = clone(fn, arg, stack, n_pages, CLONE_VM);

  if(pid < 0) {
    free(stack);
    return -1;
  }
  return pid;
}

int
thread_join(void)
{
  void *stack = 0;
  int pid = join(&stack);

  if(pid > 0 && stack != 0) {
    free(stack);
  }
  return pid;
}

```

`user/uthread.c`의 `thread_create()`와 `thread_join()` 함수를 구현하였다.

`thread_create()` 함수는 `n_pages` 만큼 동적으로 스택 메모리를 할당해주고 `clone()` 시스템 콜을 호출한다. `thread_join()` 함수는 `join()` 시스템 콜을 호출하여 종료된 스레드의 스택 포인터를 받아와 `free()`를 이용해 메모리 누수를 막는다.

Step 3: Coordinate Process-wide System Calls

```
int
kwait(uint64 addr)
{
        // ...
            for(pp = proc; pp < &proc[NPROC]; pp++){
                if(pp->parent != p || pp->group_leader != p
p) continue;
```

```
        // ...
}
```

`kwait()` 안의 자식 식별 로직을 그룹 리더만 찾도록 수정하였다.

```
void
kexit(int status)
{
    struct proc *p = myproc();

    if(p == initproc)
            panic("init exiting");

    if(p == p->group_leader) {
        acquire(&wait_lock);
        for(struct proc *pp = proc; pp < &proc[NPROC]; pp+
+) {
          if(pp->group_leader == p && pp != p) {
            acquire(&pp->lock);
            pp->killed = 1;
            if(pp->state == SLEEPING) pp->state = RUNNABLE;
            release(&pp->lock);
          }
        }
        for(struct proc *pp = proc; pp < &proc[NPROC]; pp+
+) {
          if(pp->group_leader == p && pp != p) {
            acquire(&pp->lock);
            while(pp->state != ZOMBIE && pp->state != UNUSE
D && pp->group_leader == p) {
              release(&pp->lock);
              acquire(&wait_lock);
              sleep(p, &wait_lock);
              acquire(&pp->lock);
            }
            if(pp->state == ZOMBIE) freeproc(pp);
            release(&pp->lock);
          }
```

```
        }
        release(&wait_lock);
    }
```

`kexit()` 함수가 리더에 의해 호출된 경우 모든 형제 스레드를 kill하고 동기적으로 회수한다.

```
int
kkill(int pid)
{
  struct proc *p;
  int target_tgid = -1;

  for(p = proc; p < &proc[NPROC]; p++){
    acquire(&p->lock);
    if(p->pid == pid){
      target_tgid = p->tgid;
      release(&p->lock);
      break;
    }
    release(&p->lock);
  }

  if (target_tgid == -1) return -1;

  for(p = proc; p < &proc[NPROC]; p++){
    acquire(&p->lock);
    if(p->tgid == target_tgid){
      p->killed = 1;
      if(p->state == SLEEPING) p->state = RUNNABLE;
    }
    release(&p->lock);
  }
  return 0;
}
```

`kkill()` 함수는 스레드 그룹 전체, 즉 `tgid`가 같은 모든 스레드를 `killed = 1`로 설정하여 kill을 수행한다.

```
27  int
28  kexec(char *path, char **argv)
29  {
30    char *s, *last;
31    int i, off;
32    uint64 argc, sz = 0, sp, ustack[MAXARG], stackbase;
33    struct elfhdr elf;
34    struct inode *ip;
35    struct proghdr ph;
36    pagetable_t pagetable = 0, oldpagetable;
37    struct proc *p = myproc();
38
39    if(p != p->group_leader) {
40      return -1;
41    }
42
43    drain_siblings(p);
44
45    begin_op();
```

`kernel/exec.c`의 `kexec()` 함수는 페이지 테이블을 갈아끼우기 전에 모든 형제 스레드를 종료하고 회수한다.

```
if(p != p->group_leader) {
    return -1;
}
```

리더가 아닌 스레드가 `exec()`을 호출하는 것을 막기 위해 위의 확인 작업을 추가하였다.

```
878  void
879  drain_siblings(struct proc *p)
880  {
881    acquire(&wait_lock);
882    for (struct proc *pp = proc; pp < &proc[NPROC]; pp++) {
883      if (pp->group_leader == p && pp != p) {
884        acquire(&pp->lock);
885        pp->killed = 1;
886        if (pp->state == SLEEPING) pp->state = RUNNABLE;
887        release(&pp->lock);
888      }
889    }
890    for (struct proc *pp = proc; pp < &proc[NPROC]; pp++) {
891      if (pp->group_leader == p && pp != p) {
892        acquire(&pp->lock);
893        while (pp->state != ZOMBIE && pp->state != UNUSED && pp->group_leader == p) {
894          release(&pp->lock);
895          sleep(chan: p, lk: &wait_lock);
896          acquire(&pp->lock);
897        }
898        if (pp->state == ZOMBIE) {
899          freeproc(p: pp);
900        }
901        release(&pp->lock);
902      }
903    }
904    release(&wait_lock);
905  }
906
```

실제 drain 과정은 `kernel/proc.c`의 `drain_sibling()` 함수에 구현되어 있다.

```
for(struct proc *pp = proc; pp < &proc[NPROC]; pp++) {
    if(pp->group_leader == p && pp != p) {
        acquire(&pp->lock);
        pp->killed = 1;
        if(pp->state == SLEEPING) pp->state = RUNNABLE;
        release(&pp->lock);
    }
}
```

먼저 모든 형제 스레드에 `killed` 신호를 보내고,

```
for(struct proc *pp = proc; pp < &proc[NPROC]; pp++) {
    if(pp->group_leader == p && pp != p) {
        acquire(&pp->lock);
        while(pp->state != ZOMBIE && pp->state != UNUSED &&
pp->group_leader == p) {
            release(&pp->lock);
```

```
            sleep(p, &wait_lock);
            acquire(&pp->lock);
        }
        if(pp->state == ZOMBIE) {
            freeproc(pp);
        }
        release(&pp->lock);
    }
}
```

형제 스레드들이 모두 `ZOMBIE`가 될 때까지 기다렸다가 `freeproc()`으로 안전하게 자원을 회수한다.

Results

앞서 메모리 누수 문제는 해결된 것을 확인하였고, 이제 과제 명세에 맞게 kernel-level 스레드의 기능이 정상적으로 동작하는지 확인하기 위해 커스텀 테스트코드를 다음과 같이 제작하였다.

```c
// user/threadtest.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "user/uthread.h"

int shared_val = 0;
int thread_pid = 0;

void my_thread(void *arg) {
    int val = *(int *)arg;

    shared_val = val;
    thread_pid = getpid();

    printf("[Thread] Running... My PID: %d, Received Arg: %
d\n", thread_pid, val);
```

```
    exit(0);
}

void empty_thread(void *arg) {
    exit(0);
}

int main(int argc, char *argv[]) {
    printf("=== Thread Project 2 Test Start ===\n");

    int parent_pid = getpid();
    printf("[Parent] My PID: %d\n", parent_pid);

    int arg = 100;

    int tpid = thread_create(my_thread, (void *)&arg, 1);

    if(tpid < 0){
        printf("[FAIL] thread_create failed!\n");
        exit(1);
    }

    int joined_pid = thread_join();

    if(joined_pid < 0){
        printf("[FAIL] thread_join failed!\n");
        exit(1);
    }

    printf("\n--- Validation Results ---\n");
    int success = 1;

    if (shared_val == 100) {
        printf("[PASS] Memory is properly shared between pa
rent and thread.\n");
    } else {
        printf("[FAIL] Memory is NOT shared. shared_val = %
d\n", shared_val);
```

```
        success = 0;
    }

    if (thread_pid != 0 && thread_pid != parent_pid) {
        printf("[PASS] Thread has an independent PID (Paren
t: %d, Thread: %d).\n", parent_pid, thread_pid);
    } else {
        printf("[FAIL] Thread PID is incorrect or identical
to parent. thread_pid = %d\n", thread_pid);
        success = 0;
    }

    printf("\n--- Stress Test (Checking for Memory Leaks) -
--\n");
    for(int i = 0; i < 500; i++) {
        if (thread_create(empty_thread, 0, 1) < 0) {
            printf("[FAIL] Failed to create thread at itera
tion %d. Possible memory leak.\n", i);
            success = 0;
            break;
        }
        thread_join();
    }

    if (success) {
        printf("[PASS] Successfully created and joined 500
threads. Memory deallocation looks good!\n");
        printf("\n=== ALL CUSTOM TESTS PASSED ===\n");
    } else {
        printf("\n=== SOME TESTS FAILED ===\n");
    }

    exit(0);
}
```

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ threadtest
=== Thread Project 2 Test Start ===
[Parent] My PID: 3
[Thread] Running... My PID: 4, Received Arg: 100

--- Validation Results ---
[PASS] Memory is properly shared between parent and thread.
[PASS] Thread has an independent PID (Parent: 3, Thread: 4).

--- Stress Test (Checking for Memory Leaks) ---
[PASS] Successfully created and joined 500 threads. Memory deallocation looks good!

=== ALL CUSTOM TESTS PASSED ===
$ QEMU: Terminated
```

메모리 공유가 제대로 이루어지는지, 스레드가 독립적인 PID를 갖는지, 메모리 누수 없이 자원이 회수되는지 모두 테스트를 통과하였다.

Troubleshooting

`kernel/exec.c`의 `kexec()` 함수 구현 과정에서 직접 `kexec()` 함수에 구현을 시도하였다. 테스트 결과 `proc`과 `wait_lock`에 대한 undeclared identifier 오류가 발생하였다. 이는 `proc`과 `wait_lock` 변수가 `kernel/proc.c` 내부에서만 선언되어 있어 scope 에러가 발생한 것이었다. `freeproc()` 함수 역시 마찬가지로 `proc.c` 파일에 `static`으로 선언되어 있어 같은 오류가 발생하였다.

이를 해결하기 위해 모든 변수와 함수 접근이 자유로운 `kernel/proc.c` 파일에 helper function인 `drain_sibling()`을 만들었다. 이 함수에 `kexec()`에서 수행하고자 하는 형제 스레드에 대한 drain을 모두 구현하고, `kernel/exec.c`에서는 이 helper function을 호출하는 방식으로 문제를 해결하였다.